

SKILL WEEK – 10

Session – 19

TASK 1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t signal_count = 0;

void signal_handler(int sig)
{
    signal_count++;

    if (sig == SIGINT)
        printf("\nHandler: SIGINT received (Ctrl+C)\n");

    else if (sig == SIGTERM)
        printf("Handler: SIGTERM received\n");

    else if (sig == SIGUSR1)
        printf("Handler: SIGUSR1 received\n");

    else if (sig == SIGUSR2)
        printf("Handler: SIGUSR2 received\n");

    printf("Signal count: %d\n", signal_count);
    fflush(stdout);
}

int main()
{
    struct sigaction sa;

    printf("Session 19 - Signal Handler Demonstration\n");
    printf("-----\n");
    printf("Process ID: %d\n\n", getpid());

    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    sigaction(SIGUSR2, &sa, NULL);

    printf("Four signal handlers registered successfully.\n");
    printf("Test signals using another terminal:\n");
    printf("kill -SIGUSR1 %d\n", getpid());
    printf("kill -SIGUSR2 %d\n", getpid());
    printf("kill -SIGTERM %d\n", getpid());
    printf("Press Ctrl+C to test SIGINT.\n\n");

    while (1)
    {
        printf("Program running... Signal count = %d\n", signal_count);
        sleep(3);
    }

    return 0;
}
```


TASK 2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/* SIGINT handler for the shell */
void handle_sigint(int sig)
{
    printf("\nShell: SIGINT received.\n");

    if (foreground_pid > 0)
    {
        printf("Shell: Forwarding SIGINT to foreground process %d\n",
            foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("Shell: No foreground job is running.\n");
    }

    fflush(stdout);
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    printf("Session 19 - SIGINT and Foreground Job Handling\n");
    printf("-----\n");

    /* Register SIGINT handler */
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);

    printf("Shell PID: %d\n", getpid());
    printf("Starting foreground job...\n\n");
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
sa.sa_flags = 0;

sigaction(SIGINT, &sa, NULL);

printf("Shell PID: %d\n", getpid());
printf("Starting foreground job...\n\n");

pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    /*
     * Child uses the default SIGINT behavior.
     * Therefore Ctrl+C can terminate the foreground job.
     */
    signal(SIGINT, SIG_DFL);

    printf("Foreground job started. PID: %d\n", getpid());
    printf("Press Ctrl+C to interrupt the foreground job.\n");

    while (1)
    {
        printf("Foreground job running...\n");
        sleep(2);
    }
}
else
{
    foreground_pid = pid;

    printf("Shell: Foreground PID = %d\n", foreground_pid);
    printf("Shell is protected from Ctrl+C.\n\n");

    waitpid(pid, &status, 0);

    if (WIFSIGNALED(status))
    {
        printf("\nShell: Foreground job terminated by signal %d.\n",
            WTERMSIG(status));
    }
    else
    {
        printf("\nShell: Foreground job completed normally.\n");
    }

    foreground_pid = -1;

    printf("Shell: Foreground job state updated.\n");
    printf("Shell: Ready for next command.\n");
}

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session19_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session19_task2.c -o session19_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session19_task2
Session 19 - SIGINT and Foreground Job Handling
-----
Shell PID: 515
Starting foreground job...

Shell: Foreground PID = 516
Shell is protected from Ctrl+C.

Foreground job started. PID: 516
Press Ctrl+C to interrupt the foreground job.
Foreground job running...
Foreground job running...
Foreground job running...
^C
Shell: SIGINT received.
Shell: Forwarding SIGINT to foreground process 516

Shell: Foreground job completed normally.
Shell: Foreground job state updated.
Shell: Ready for next command.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

Session – 20

TASK 1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;
int job_stopped = 0;

/* Handle Ctrl+Z */
void handle_sigstsp(int sig)
{
    printf("\nShell: SIGTSTP received.\n");

    if (foreground_pid > 0)
    {
        printf("Shell: Suspending foreground process %d\n",
            foreground_pid);

        kill(foreground_pid, SIGSTOP);

        job_stopped = 1;

        printf("Job state: STOPPED\n");
        printf("Job state preserved for later resume.\n");
    }
    else
    {
        printf("Shell: No foreground job is running.\n");
    }

    fflush(stdout);
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    printf("Session 20 - SIGTSTP and Job Suspension\n");
    printf("-----\n");

    /* Register SIGTSTP handler */
    sa.sa_handler = handle_sigstsp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGTSTP, &sa, NULL);

    printf("Shell PID: %d\n", getpid());
    printf("Starting foreground job...\n\n");
```

[illegible]

```
keerthi_priya@LAPTOP-OSGV X Ubuntu X + v
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Job removed from job table.
Session completed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$
```

TASK 2: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
keerthi_priya@LAPTOP-OSGV × Ubuntu
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    pid_t shell_pgid;
    pid_t child_pgid;
    int status;

    printf("Session 20 - Process Group Management\n");
    printf("-----\n");

    shell_pgid = getpgrp();

    printf("Shell PID      : %d\n", getpid());
    printf("Shell PGID     : %d\n", shell_pgid);
    printf("Terminal PGID  : %d\n\n", tcgetpgrp(STDIN_FILENO));

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        /* Child creates a new process group */
        setpgid(0, 0);

        child_pgid = getpgrp();

        printf("Child PID      : %d\n", getpid());
        printf("Child PGID     : %d\n", child_pgid);

        printf("Child: Process group created successfully.\n");
        printf("Child: Waiting for signal...\n");

        while (1)
        {
            pause();
        }
    }
    else
    {
        /* Put child into its own process group */
        setpgid(pid, pid);
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
}
}
else
{
    /* Put child into its own process group */
    setpgid(pid, pid);

    child_pgid = getpgid(pid);

    printf("Parent: Child PID = %d\n", pid);
    printf("Parent: Child PGID = %d\n", child_pgid);

    /*
     * Transfer terminal control to child process group.
     */
    if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("Parent: Terminal control transferred to child group.\n");
    }

    sleep(2);

    printf("\nParent: Sending SIGTERM to child process group.\n");

    /* Send signal to entire process group */
    kill(-child_pgid, SIGTERM);

    waitpid(pid, &status, 0);

    /*
     * Restore terminal control to the parent.
     */
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("Parent: Terminal control restored.\n");
    }

    printf("Parent: Child process group terminated.\n");
    printf("Parent: Process group management completed.\n");
}

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session20_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session20_task2.c -o session20_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session20_task2
Session 20 - Process Group Management
-----
Shell PID      : 821
Shell PGID     : 821
Terminal PGID  : 821

Parent: Child PID = 822
Parent: Child PGID = 822
Parent: Terminal control transferred to child group.
Child PID      : 822
Child PGID     : 822
Child: Process group created successfully.
Child: Waiting for signal...

Parent: Sending SIGTERM to child process group.

[1]+  Stopped                  ./session20_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```